Vision transformers and multimodal retrieval

COCO

LECTURE 23 | GÉRON CH. 15–16

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Part V built retrieval over text and over user behaviour. Both reduced to the same object: **two encoders and an inner product**.

- Lecture 20 — a query encoder and a document encoder, retrieved by inner product
- Lecture 22 — a user tower and an item tower, the same architecture with users in place of queries, and an evaluation whose number depends on the protocol stated beside it

> **TODAY THE TWO TOWERS TAKE DIFFERENT MODALITIES**
>
> One encodes **images**, the other **text**, and a photograph and its caption must land close together. Everything about the retrieval is unchanged; what is new is that the two encoders never see the same kind of input.

Today

1. The brief: a catalogue of photographs searched with sentences, and the vision transformer, which reads an image as a sequence of patches
2. A metric and its baseline: Recall@k, the mean reciprocal rank, and what random ranking scores, exactly
3. Two encoders trained separately, measured against each other
4. **The mathematics** — why the sphere, the gap between the modalities, and what the temperature does
5. A jointly trained dual encoder: a known-answer test, text-to-image retrieval, and an audit of the evaluation
6. **Design exercise** — a stock-photo company's text-to-image search, in nine steps, with the pitfalls and how to address them

Open Lecture 23 in Colab

The stakeholder

A retailer’s catalogue team. Their catalogue has one property that breaks every search system they have bought:

> **THE ENTRIES ARE PHOTOGRAPHS**
>
> Product photos arrive from suppliers with an identifier, a price and a picture. Written descriptions are inconsistent, often absent, and never written by the person searching.

Customers type sentences. The catalogue stores pixels. Nothing in the system converts one into the other.

What they are asking for

Input	Output
A sentence typed by a customer	A ranked list of catalogue entries
“a red bus on a city street”	the entries whose *picture* shows that

- No labels. Nobody is going to annotate the catalogue
- No fixed vocabulary. The query is whatever the customer typed
- The answer must be a **ranking**, not a yes/no

This is the first brief in the course whose input and output are not the same kind of object.

Why the obvious thing does not apply

Keyword search needs text on both sides. So does the semantic search you built in the previous application: it embeds a sentence and compares it to other *sentences*.

> **THERE IS NO TEXT ON THE CATALOGUE SIDE**
>
> You can index descriptions when they exist. But an entry with no description is invisible to a text index, no matter how good the index is — not badly ranked, absent.

Hold that thought. The next lecture counts how many entries it costs us, and repairs it.

The corpus, and its size

200

catalogue entries
32.6 MB on disk

- COCO 2014 validation images, taken in order of image id, so everyone's catalogue is the same catalogue
- 5 independent human captions per image — written by people who were not shown each other's
- Every retrieval number today is measured against **200 candidates**

✗ Say the candidate-set size every time you say a recall. A recall without it is not a measurement.

> **FAILURE CONDITION — THE CANDIDATE SET IS THE NUMBER**
>
> Every retrieval score here is measured against 200 candidates. Widen the catalogue and the same model reports a worse number without having changed, so a recall figure quoted without its candidate-set size cannot be compared with anything.

Sixteen of the two hundred entries

16 of the 200 catalogue entries — COCO validation images, each with five human captions we hold back



Ordinary photographs of ordinary things — and five sentences each, written by five different people, which the search system never sees.

Two different uses for five captions

Caption	Role	Who sees it
#1	the entry's *description*, when it has one	the text index
#2	the customer's *query*	the search box
#3–5	held back	nobody, today

Using two different people's sentences about the same picture makes this a paraphrase problem rather than a lookup. If we used the same sentence on both sides, a hash table would score 100%.

A second corpus, for a different job

Test against a case whose answer you know. The catalogue has no labels, so it cannot play that role.

Corpus	Size	Job
COCO catalogue	200 images	the real task: rank entries for a query
CIFAR-10 test	500 images	the known-answer check: ten labels we have

CIFAR-10 is 32×32. That is a hard setting for any model trained on web photographs, and we will see the cost.

An image as a sequence

A transformer takes a sequence. An image is a grid. So cut it up:

1. Split the image into fixed patches — 16×16 pixels is the usual choice
2. Flatten each patch and project it linearly to d_{model}. That projection plays the part of a language model's token-embedding table, with patches in place of tokens
3. Add a positional encoding, because attention by itself has no notion of order — and a grid has two
4. Run the standard transformer blocks

A 224×224 image at 16×16 patches is $14 \times 14 = 196$ tokens. Well within reach of the quadratic cost, which is exactly why this patch size and not a smaller one.

What would “it works” even mean?

The system returns a ranked list. So the quantity that matters is **where the right entry landed**.

$$r_q \;=\; 1 + \#\{\, j \neq q^\star : s(q, j) \geq s(q, q^\star) \,\}$$

THE RANK OF THE RELEVANT ENTRY FOR QUERY Q

Ties count *against* us: a tie is scored as the worse rank. A metric that flatters a degenerate score matrix is not a metric.

Recall@k

$$\mathrm{R@}k \;=\; \frac{1}{Q}\sum_{q=1}^{Q}\mathbf{1}\left[\,r_q \le k\,\right]$$

THE SHARE OF QUERIES WHOSE ANSWER IS IN THE TOP K

With exactly one relevant entry per query, recall and precision at *k* carry the same information, and recall is the one the literature reports. We will read R@1, R@5 and R@10.

R@1 is the number the product actually cares about: it is the top of the page.

Mean reciprocal rank

$$\mathrm{MRR} = \frac{1}{Q}\sum_{q=1}^{Q}\frac{1}{r_q}$$

Rank 1 contributes 1, rank 2 contributes 0.5, rank 100 contributes 0.01. It rewards being nearly right and stops rewarding it quickly — which matches a page of ten results.

The trivial baseline, computed rather than run

Every number needs a baseline. Here the baseline needs no experiment at all.

Shuffle the catalogue. One relevant entry among n, placed uniformly at random:

$$\Pr[r \le k] = \frac{k}{n}, \qquad \mathbb{E}[r] = \frac{n+1}{2}, \qquad \mathbb{E}\left[\frac{1}{r}\right] = \frac{H_n}{n}$$

RANDOM RANKING, EXACTLY

On our catalogue, n = 200: R@1 is 0.5%, R@10 is 5.0%, the expected rank is 100.5 and the MRR is 0.029.

H_n is the harmonic number. No simulation, no seed, no error bar — this one is arithmetic.

Step 1 — encode the images

```python
# an image-only encoder: ImageNet labels, no text anywhere in training
import torch
from transformers import AutoImageProcessor, ViTModel

proc = AutoImageProcessor.from_pretrained("google/vit-base-patch16-224")
vit = ViTModel.from_pretrained("google/vit-base-patch16-224",
                               add_pooling_layer=False).to(device).eval()

feats = []
with torch.no_grad():
    for i in range(0, len(images), 32):
        batch = proc(images=images[i:i + 32], return_tensors="pt").to(device)
        feats.append(vit(**batch).last_hidden_state[:, 0].cpu())  # [CLS]

V = torch.cat(feats).numpy()
assert V.shape == (200, 768), V.shape
```

`add_pooling_layer=False` because that checkpoint has no pooler weights — ask for one and you get a randomly initialised layer, silently.

Step 2 — encode the queries

```python
from transformers import AutoModel, AutoTokenizer

tok = AutoTokenizer.from_pretrained("sentence-transformers/all-MiniLM-L6-v2")
enc = AutoModel.from_pretrained("sentence-transformers/all-MiniLM-L6-v2") \
                .to(device).eval()

with torch.no_grad():
    b = tok(queries, return_tensors="pt", padding=True,
            truncation=True, max_length=128).to(device)
    h = enc(**b).last_hidden_state                # (200, tokens, 384)
    m = b["attention_mask"].unsqueeze(-1).float()
    T = ((h * m).sum(1) / m.sum(1)).cpu().numpy()   # mean pooling

assert T.shape == (200, 384), T.shape
```

Mean pooling over real tokens only — the mask matters, and forgetting it averages in the padding.

Step 3 — make the shapes agree, then measure

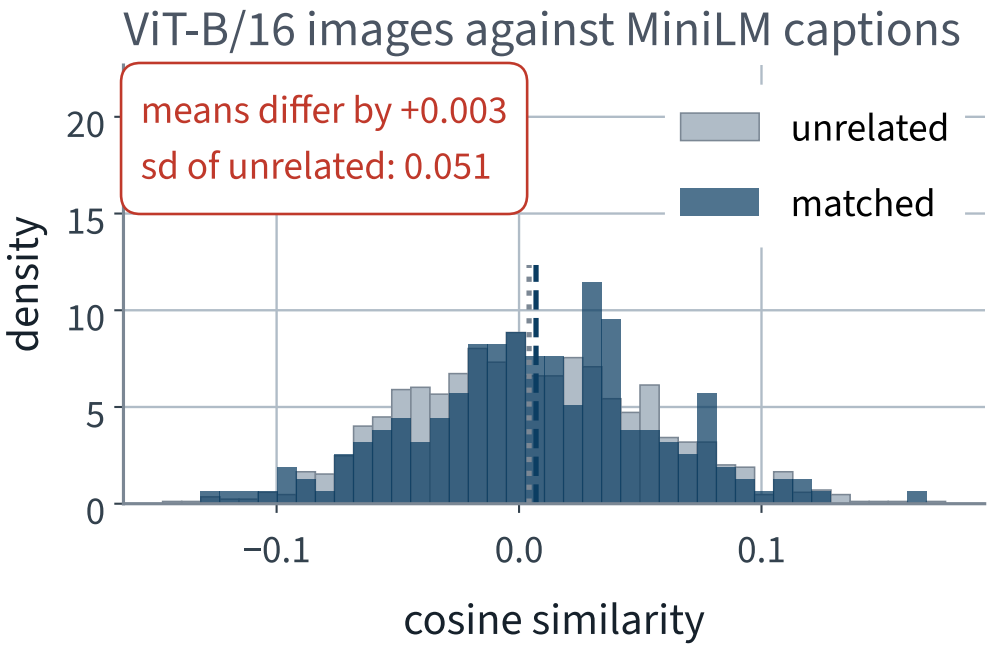
The image vectors have 768 coordinates and the sentence vectors 384, so `V @ T.T` does not even run. Make them agree three ways, so that nobody can blame the fix: a random Gaussian projection 768 → 384, the first 384 image coordinates, or the text zero-padded to 768.

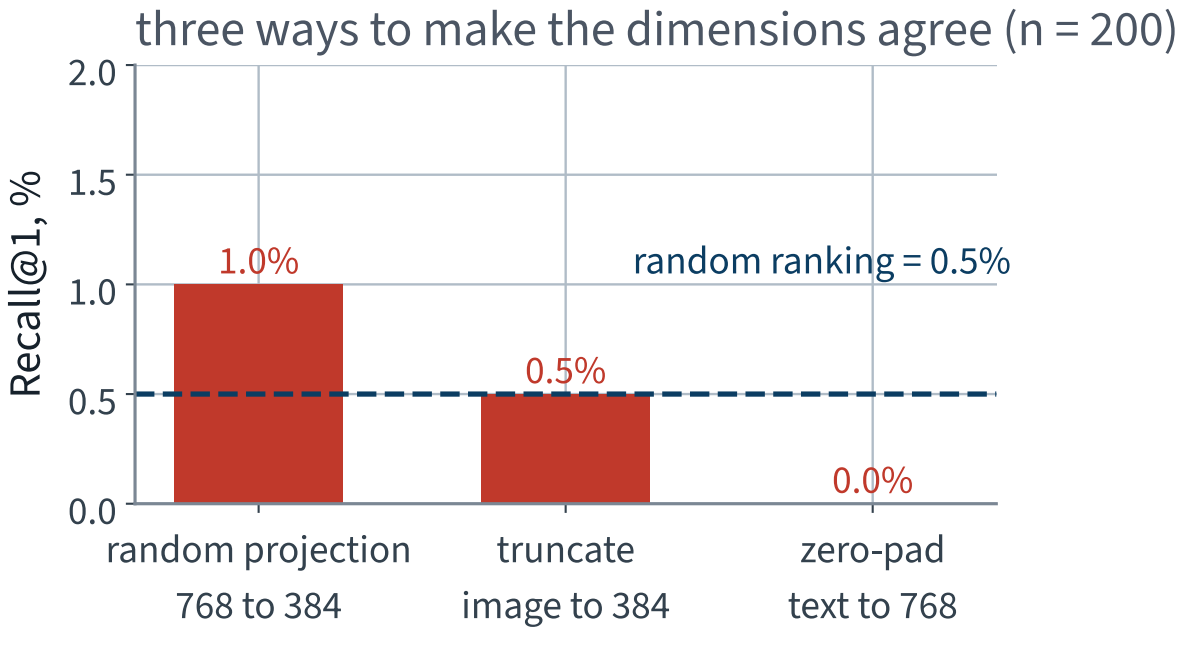
Then measure — against what? Compute two numbers, always:

- the cosine between an image and **its own caption**
- the cosine between an image and **somebody else's caption** — 200 × 200 minus the diagonal, so nearly forty thousand of them

> **THE COMPARISON IS THE MEASUREMENT**
>
> If the two distributions sit on top of one another, the similarity carries no information about whether the pair matches.

The two spaces, measured





The matched pairs and the unrelated pairs are the same distribution, and all three fixes land on the random-ranking line.

And as a retrieval system

System	R@1	R@5	R@10	Median rank
Random ranking (exact)	0.5%	2.5%	5.0%	100
ViT + MiniLM, random projection	1.0%	4.0%	6.0%	97
ViT + MiniLM, truncated	0.5%	2.0%	5.5%	88
ViT + MiniLM, zero-padded	0.0%	2.5%	5.0%	88

✗ Two of the 200 queries put the right image first. Shuffling is expected to get one. Two strong encoders, over a hundred million parameters between them, and that is the whole difference.

Say the result out loud

A picture of a bus and the sentence *“a red bus on a city street”* are **no closer** than that picture and a sentence about a sandwich.

BOTH ENCODERS WORK PERFECTLY

The ViT separates images from one another. MiniLM separates sentences from one another. Neither was ever shown a single (image, sentence) pair, so neither has any reason to place them anywhere in particular relative to each other.

Why, precisely

- An embedding space is defined *only* up to whatever the training objective constrained
- ImageNet classification constrains images relative to images
- Sentence-similarity training constrains sentences relative to sentences
- Nothing in either loss ever compared a vector from one to a vector from the other, so the relative orientation of the two spaces is **arbitrary**

> **A ROTATION YOU CANNOT SEE**
>
> Apply any orthogonal map to the image space: every image–image cosine is unchanged, so the ViT's loss is unchanged. The image–text cosines all change. The loss cannot distinguish those worlds, so it did not pick one.

Why normalise at all

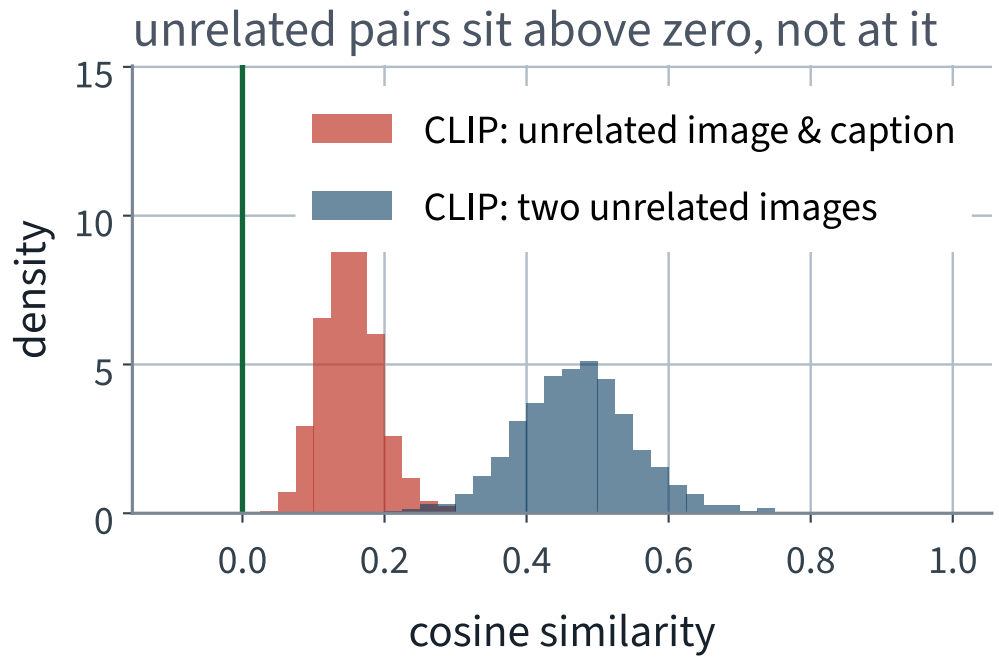
The unnormalised inner product factorises:

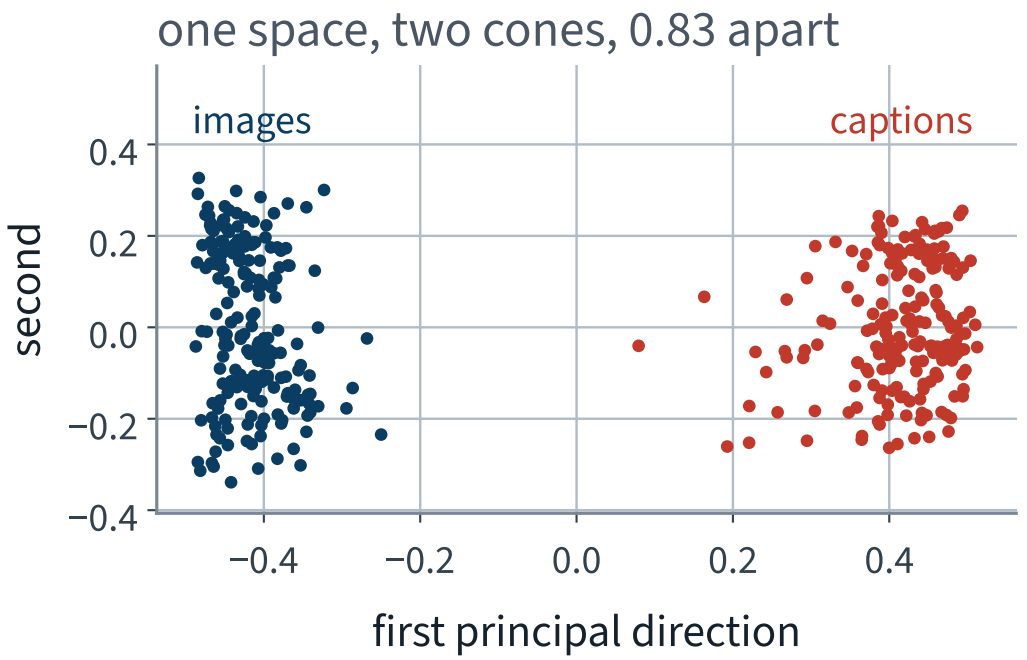
$$\langle a, b \rangle = \|a\| \, \|b\| \cos\theta$$

- Two quantities are mixed: *which direction* the encoder chose, and *how loudly* it said it
- Only the direction carries the semantics. The length is whatever the last linear layer happened to scale to
- Measured on our catalogue, with the jointly trained encoder used later today: the longest image embedding is 1.32× the shortest

Divide both by their norms and the score $S_{ij} = \mathbf{u}_i^\top \mathbf{v}_j$ of image i against caption j is a pure cosine, in $[-1, 1]$, comparable across pairs, models and runs.

One space, two cones





In the jointly trained encoder used later today, images occupy one region of the sphere and captions another: the two centroids are 0.83 apart. The objective aligns pairs *relatively*, and never asks the two clouds to coincide.

What dividing by τ actually does

Training scores a batch of B pairs. Row i of S is a B-class softmax whose correct answer is column i, with every cosine divided by a temperature τ — the InfoNCE loss:

$$\mathcal{L}_i = -\log \frac{\exp(S_{ii}/\tau)}{\sum_j \exp(S_{ij}/\tau)}, \qquad \frac{\partial \mathcal{L}_i}{\partial S_{ij}} = \frac{1}{\tau}\left(p_{ij} - \mathbf{1}[j = i]\right)$$

THE SOFTMAX OF ROW I, PREDICTION MINUS TARGET, SCALED BY THE INVERSE TEMPERATURE

- τ does **not** change which column is largest, so it cannot change the top-1 accuracy of a fixed model
- It changes *where the gradient goes*. Small τ concentrates p on the few hardest negatives; those get almost all the push
- Too small and the loss is dominated by one negative per row, which may be a mislabelled pair rather than an informative one

The idea, in one paragraph

Take a large collection of (image, caption) pairs. Push each image through an image tower and each caption through a text tower, both ending in a linear map into the *same* d-dimensional space. Normalise onto the unit sphere. Then train so that, within a batch, each image's own caption scores higher than every other caption in the batch.

The checkpoint we use: a ViT-B/32 image tower and a 12-layer text transformer, meeting in one 512-dimensional space on the unit sphere, trained on image–text pairs collected from the web.

> **THE DERIVATION, EARLIER TODAY**
>
> Why the sphere, why the two modalities still sit in separate regions of it, and what the temperature does — all from this lecture's mathematics block. Here we use the result and measure it.

The whole system, in fourteen lines

```python
from transformers import CLIPModel, CLIPProcessor

proc  = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32") \
                 .to(device).eval()

with torch.no_grad():
    ib = proc(images=images, return_tensors="pt").to(device)
    I  = model.get_image_features(**ib).cpu().numpy()
    tb = proc(text=queries, return_tensors="pt",
              padding=True, truncation=True, max_length=77).to(device)
    T  = model.get_text_features(**tb).cpu().numpy()

I, T = unit(I), unit(T)          # onto the sphere, both of them
sim  = T @ I.T                   # (queries, images) — now this is legal
assert sim.shape == (200, 200)
```

Same shape on both sides, so the matrix product exists. That is the entire difference.

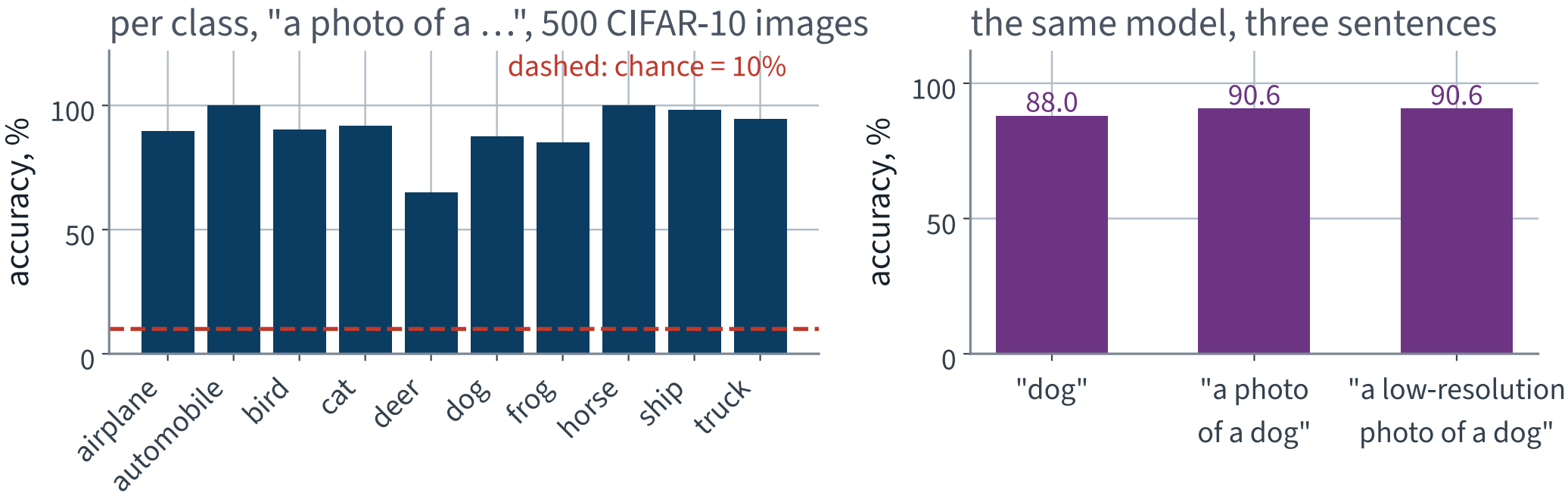
Before the catalogue: the known-answer test

The catalogue has no labels, so nothing there can tell us the model is loaded correctly, pointed at the right tensors, and preprocessing the images the way it was trained to.

- Take 500 CIFAR-10 test images and the ten class names
- Embed the sentence *“a photo of a {class}”* for each class
- Assign each image to the nearest of the ten sentences

No training, no fitting, no labels used except to score. This is what “zero-shot” means, and it is the check, not the product.

Zero-shot on a set whose labels we have



90.6% over 500 images against a 10% chance level — from a model that has never seen CIFAR-10.

The sentence is a hyperparameter

Prompt	Accuracy over 500 images
“dog”	88.0%
“a photo of a dog”	90.6%
“a low-resolution photo of a dog”	90.6%
Chance	10.0%

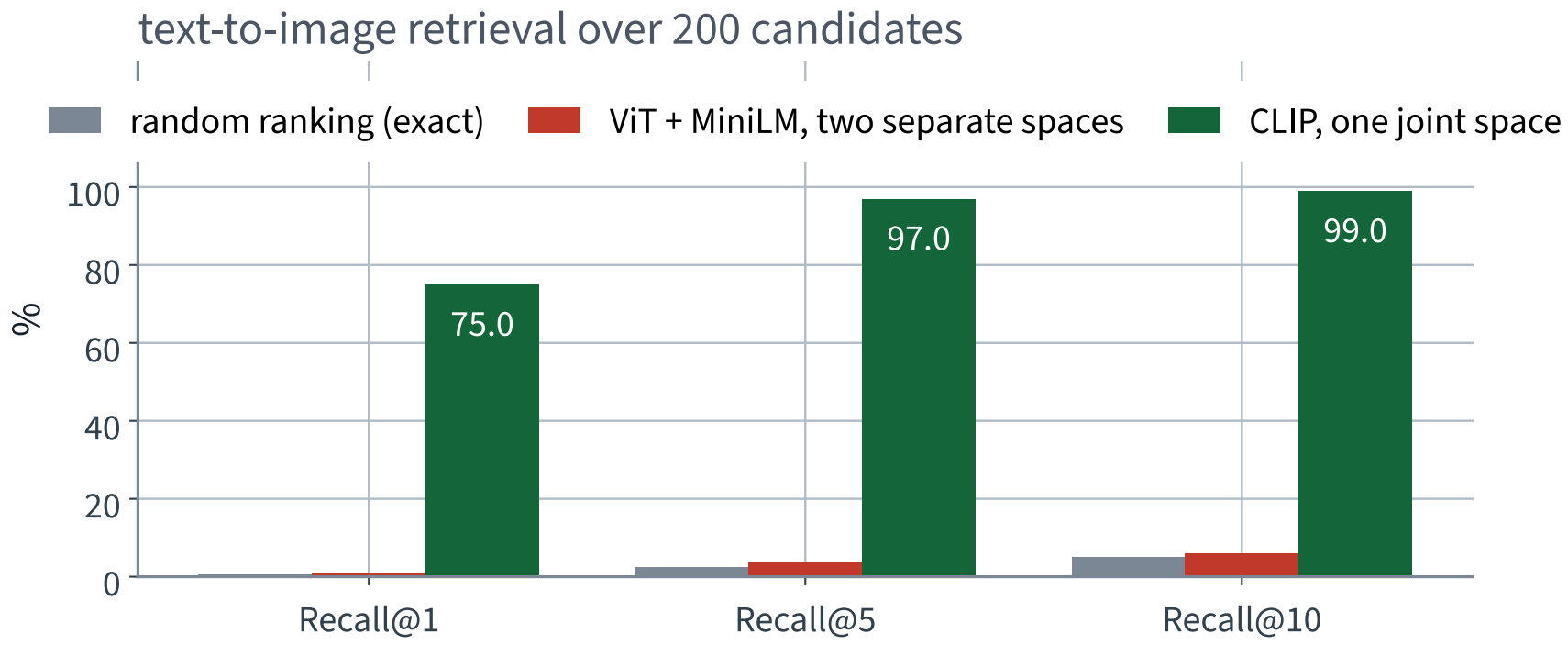
✗ A spread of 2.6% from changing the wording alone. A “zero-shot” number is a number for one particular sentence, and the sentence is a choice you made.

The check passed. Back to the catalogue

Same 200 images. Same 200 queries. Same metric, same ties rule, same baseline. Only the encoders changed.

```
rank = (sim >= sim.diagonal()[:, None]).sum(axis=1)   # ties count against us
for k in (1, 5, 10):
    print(f"R@{k:<3d} {(rank <= k).mean():6.1%}   random: {k / len(rank):6.1%}")
print(f"median rank {np.median(rank):.0f} of {len(rank)}")
```

Three systems, one metric



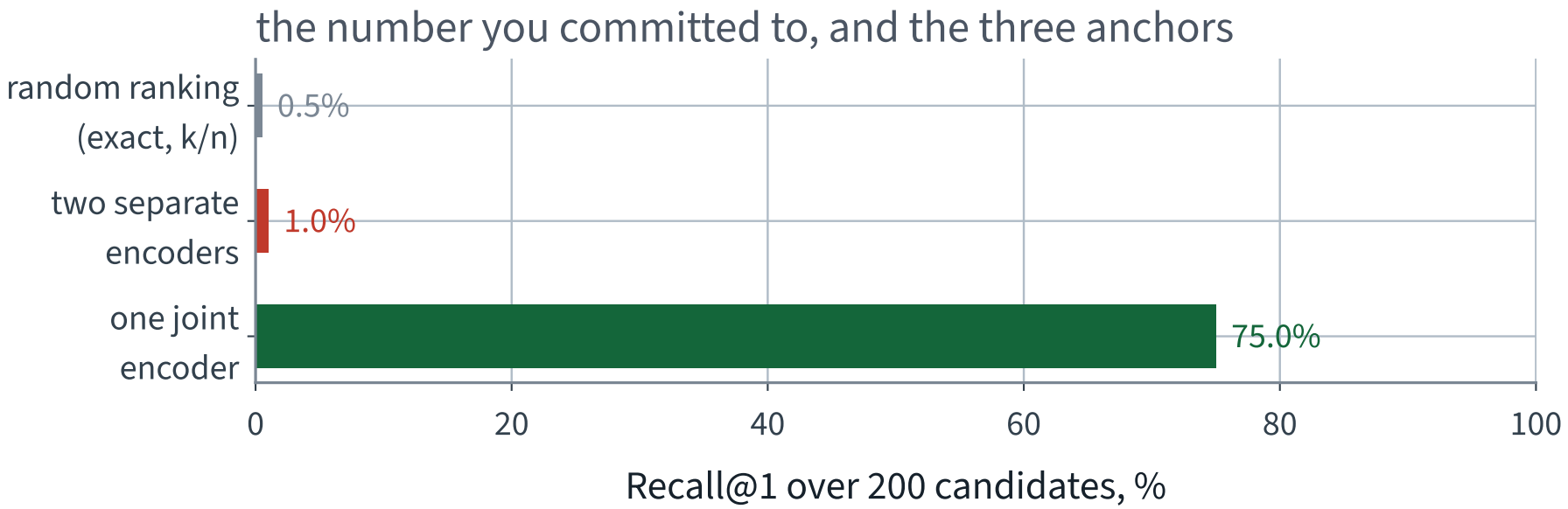
Random ranking and the two-encoder system are the same bar. The joint encoder is a different picture entirely.

The results

System	R@1	R@5	R@10	Median rank	MRR
Random ranking	0.5%	2.5%	5.0%	100	0.029
ViT + MiniLM	**1.0% ✗**	4.0%	6.0%	97	0.037
Joint dual encoder	**75.0% ✓**	**97.0% ✓**	**99.0% ✓**	1	0.841

Text to image, 200 candidates, ties against the model, one caption per entry as the query.

Recall@1, against the anchors



Two of these three bars are the same bar. Which two, and why?

Report the failure cases

75.0% at rank 1 means **25.0%** of queries did not get the right entry first. Look at them.

- Queries that describe a scene rather than an object — several entries fit equally well, and the “wrong” one is often a reasonable answer
- Fine distinctions the caption makes and the picture barely shows
- Counts and spatial relations: “three”, “behind”, “to the left of”

> **ONE RELEVANT ENTRY IS A FICTION**
>
> Our metric assumes exactly one right answer per query because that is what the data gives us. For “a man riding a horse” on a real catalogue it is simply false, and the measured recall is therefore a *lower* bound.

The queries that have no single answer

Customers do not type captions. They type things like:

- “something to sit on”
- “gear for bad weather”
- “something you would take to the beach”

The right output is not one entry. It is a shortlist with reasons — and we have no way to produce, or to score, a reason.

The second thing the next lecture repairs.

Six ways to leak in a retrieval evaluation

1. The query text is also the indexed description — you are measuring string equality
2. The query's own entry is scored against a shortlist that was built using the query
3. The prompt template was chosen by looking at the number it produced
4. Recall reported without the candidate-set size
5. Ties broken in the model's favour — a constant scorer then reports R@1 = 100%
6. Normalisation applied to one side only, so "cosine" is not a cosine

Audit this

```
sim  = T_emb @ I_emb.T
rank = 1 + (sim > sim.diagonal()[:, None]).sum(axis=1)
print(f"R@1 = {(rank == 1).mean():.1%}")
```

It runs. On the real embeddings it prints a number close to the R@1 in the results table. What is wrong with it, and what input would make it print 100%?

Answer

> **STRICT INEQUALITY**
>
> `>` counts only candidates that *beat* the right answer. Anything that ties with it is placed behind it, for free.

Feed it a similarity matrix of all zeros — a model that has learned nothing, or a bug that zeroed the embeddings — and every rank is 1. It reports **R@1 = 100%**.

```
rank = (sim >= sim.diagonal()[:, None]).sum(axis=1)   # ties count against us
```

Audit this too

```
best = None
for template in ["{}", "a photo of a {}", "a low-resolution photo of a {}"]:
    acc = zero_shot_accuracy(template, X_test, y_test)
    if best is None or acc > best[1]:
        best = (template, acc)

print(f"best template {best[0]!r}: {best[1]:.1%} on the test set")
```

What is wrong with it, and where have you met the same error before?

Answer

SELECTION ON THE TEST SET

Three templates were tried and the best *on the test set* was reported. That is a hyperparameter chosen on the test set — the same error, with a prompt template in place of a numeric hyperparameter.

The spread across the three was 2.6%, so the optimism is of that order. The repair is the one you already know: choose the template on a validation split, report the chosen one on the test set, once.

The notebook for this lecture

Open Lecture 23 in Colab

- **Runs on free CPU.** Pretrained encoders, inference only, on the 200-entry catalogue
- The two separately trained encoders, their dimensions matched three ways, measured against the matched-versus-unrelated control
- The jointly trained encoder: its embedding lengths, the gap between the modalities, the CIFAR-10 known-answer test, and the recall table

> **WHAT TO CHANGE**
>
> Skip the normalisation before the inner product. Retrieval degrades and the reason is in the derivation: without unit norm the score confounds direction with magnitude, and magnitude carries no meaning here.

Design exercise · in the manner of an ML system-design interview

The task: a stock-photo company’s text-to-image search

> **THE BRIEF**
>
> The company licenses photographs to designers; its search page is where every licence starts. A designer types “two colleagues laughing over coffee” and the page must return a **ranked page of photographs** whose pixels show that. Contributors upload with a caption and keywords written to be found, not to be accurate; the designer’s words are never the contributor’s. The business measures downloads per search and licence revenue.

- **Given:** the library — photographs, illustrations and vector art, each with its contributor’s caption and keywords — the editors’ category labels on a subset, the search log with its downloads
- **Scale:** tens of millions of images, uploads every minute, a page of twenty results per query
- **Consumers of the output:** the search page and the editors who curate the front page

✓ Nine steps follow, each with the pitfall that catches a first attempt and what to do about it. The model at the centre is today’s: an image tower and a text tower trained jointly on one sphere.

Step 1 — Frame it: the rank of the right picture, over a stated number of candidates

- The output is a ranking, so the quantity is where the relevant picture landed: recall at a cut-off, with the cut-off at the page the designer sees, and the mean reciprocal rank beside it
- Ties count against the model: a constant scorer must report what shuffling reports
- The baseline is arithmetic, not an experiment: for one relevant picture among n, recall at k under random ranking is k/n

> **PITFALL — THE CANDIDATE SET IS THE NUMBER**
>
> A recall quoted without the size of the set it was measured against cannot be compared with anything. Three in four at rank one over a few hundred candidates is not three in four over the library, and the same model reports a worse number as the haystack grows without having changed, so a launch promised on the small number disappoints. Say the size every time you say a recall, and measure at the library's size.

Step 2 — The data: image–caption pairs, and what a caption is for

- The training signal is a pair: a picture and a sentence that describes it. The library has millions of them for free — the contributors' captions
- But a contributor's caption is written to be found: a list of keywords, the brand of the camera, a mood the picture does not show, the same sentence on every frame of a shoot
- For the judged queries, the editors write what a designer would type; the search log supplies real queries, with the downloads that followed them as judgements

> **PITFALL**
>
> Measuring with the contributor's own caption as the query. The text index already holds that string, so the system is scored on string equality and a hash table would be perfect. Use a different person's sentence about the same picture on the query side, so that the task is paraphrase rather than lookup; and clean the keyword-stuffed captions before pairing, because the model learns what the pairs say.

Step 3 — Split by shoot, before anything is fitted

- Hold out the judged queries and their pictures as the test set; carve a validation set from the rest for every choice — prompt wording, temperature, cut-off
- A library is organised in shoots: dozens of near-identical frames of the same scene, same caption, same contributor. Split by shoot, so that no frame of a test scene is in the training pairs
- Read the test set once, after everything is chosen

> **PITFALL**
>
> Splitting by picture. Another frame of the same shoot, with the same caption, sits in the training pairs, and the model has seen the test answer up to a change of angle: a leak of the standing kind, with no column duplicated. The recall is splendid, it measures memory of the shoot, and the review approves an untested model. Group by shoot and contributor, and split the groups.

Step 4 — Two encoders, one space, on the unit sphere

- The image tower is a vision transformer: the picture cut into patches, each projected like a token, a positional encoding, the standard blocks. The text tower is a transformer over the words
- Both end in a linear map into the same space, and both outputs are normalised, so that the score is a pure cosine: direction carries the meaning, length is whatever the last layer scaled to
- Precompute every picture's vector once; at query time, encode the sentence and take the inner products

> **PITFALL**
>
> Two strong encoders trained apart — a vision transformer from image labels, a sentence encoder from sentence pairs — glued together by matching their dimensions. Nothing in either loss ever compared a vector from one with a vector from the other, so their relative orientation is arbitrary and the retrieval is indistinguishable from shuffling. The fix is not a bigger encoder; it is a training signal that saw both at once.

Step 5 — The contrastive objective: the batch is where the negatives come from

- A batch of B pairs gives a $B \times B$ matrix of cosines; row i is a B-class softmax whose right answer is column i. Every other caption in the batch is a negative, for free
- The cosines are divided by a temperature τ before the softmax. It cannot change which column is largest; it decides where the gradient goes — small τ sends almost all of it to the few hardest negatives
- Choose τ on the validation split, with the batch composition held fixed

PITFALL — A FALSE NEGATIVE IN EVERY BATCH

Batches drawn in upload order hold several frames of the same shoot, each with the same caption. The objective then pushes a picture away from a sentence that describes it, and with a small τ that one mislabelled pair takes the whole row's gradient. Draw batches across shoots, one frame per shoot, and keep τ away from the floor where a single negative dominates.

Step 6 — A known-answer test before the catalogue is trusted

- The library has no labels that say “this is the right picture”. A subset has the editors’ category labels, and that is the known answer: embed “a photo of a {category}” for each and assign every labelled picture to the nearest sentence
- No fitted parameter anywhere: the check that the model is loaded, pointed at the right tensors, and preprocessing pictures as it was trained to
- Read the accuracy per category and per content type — photograph, illustration, vector art

PITFALL — THE SENTENCE IS A HYPERPARAMETER

Trying several wordings and reporting the best on the test labels: a hyperparameter chosen on the test set, with a prompt in place of a number, and the spread between wordings is the size of the optimism. Choose the wording on the validation split and report it on the test once. And a checkpoint that silently loads a randomly initialised head still prints a number; the known-answer test catches it before a customer does.

Step 7 — Recall on the judged queries, sliced by what the library holds

- Same metric, same ties rule, same random baseline, on the editors' judged queries over a candidate set the size of the library's: recall at the page and the reciprocal rank, read once
- The model was pretrained on web photographs and captions. The library is studio shots on white, illustrations, vector art, and queries about mood that no web caption contains
- Report the failure cases: scenes with several reasonable answers, fine distinctions, counts and spatial relations

PITFALL

One mean over a web-like benchmark, carried to the library. Slice the recall by content type and by query type: a model that puts the right photograph first three times in four can rank illustrations no better than chance, the mean hides it, and illustration downloads fall. And one relevant picture per query is a fiction on a real library, so the measured recall is a lower bound — judge several, and say so.

Step 8 — In production: the release, the refresh, and who is retrieved

- The index is the matrix of picture vectors and the model version that produced them. A new upload is one encoder pass; a new model re-encodes the library, built beside the old index and switched once, behind an A/B test on downloads per search
- Re-run the known-answer test and the judged queries after every rebuild; a drop is the regression, owned by whoever is on call
- Audit a fixed set of queries about people — “a doctor”, “a family” — and record who the first page shows

PITFALL — WHO THE MODEL RETRIEVES

The pairs the model learned from came from the web, whose captions attach some people to some words. Recall gives full credit for a first page that answers “a doctor” with one kind of person every time, nothing in the objective notices, and the customer who does is a journalist. Measure it as a distribution over the first page for the audit queries, a requirement the editors sign off, re-checked on every refresh.

Step 9 — Ship it: the pipeline, and the pitfalls in one table

Uploads with captions → captions cleaned, editors' queries on the query side → split by shoot → two towers into one space, both normalised → contrastive training, batches across shoots → known-answer test → recall at the page on the judged queries, sliced by content type → A/B test, index with its model version, audit queries re-run.

Pitfall	Address it by
Recall without its candidate-set size	the size in the sentence; the baseline computed
The caption as the query	another person's sentence as the query; keywords cleaned
Frames of a test shoot in training	splitting by shoot and contributor
Two encoders trained apart	a jointly trained pair on one sphere, both sides normalised
A shoot's frames as negatives	batches across shoots; τ off the floor
The prompt chosen on the test labels	wording chosen on validation; the known-answer test first
A web benchmark read as the library	recall sliced by content and query type

What we did

1. Fed an image to a transformer as a sequence of patches
2. Measured retrieval by the rank of the right entry — Recall@k and the mean reciprocal rank, ties counted against the model, beside a random baseline computed exactly
3. Encoded images and text with two separately trained models and found their spaces unrelated: nothing in either loss ever compared a vector from one with a vector from the other
4. Derived why embeddings are normalised onto the sphere, saw that the two modalities keep separate regions of it, and what τ does: it cannot reorder a row, it decides where the gradient goes
5. Used a jointly trained dual encoder — a known-answer test first, then text-to-image retrieval at 75.0% R@1 over 200 candidates
6. Audited the evaluation: the failure cases, six ways to leak, the tie bug and the template chosen on the test set
7. Designed a stock-photo company's text-to-image search in nine steps — recall with its candidate-set size, another person's sentence as the query, a split by shoot, two towers on one sphere, batches drawn across shoots with τ off the floor, a known-answer test with its wording chosen on validation, recall sliced by content type, and a release behind an A/B test with the index versioned and an audit of who is retrieved — each step named for the pitfall it avoids

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 23

five, in the style of the written paper

Exercises — Lecture 23

1 Two encoders are trained separately, one on images and one on text. Explain why their cosine similarity carries no information about matching, using a rotation argument. [5]

2 A rank is computed by counting only the candidates that score strictly higher than the right answer. State the Recall@1 this reports for a similarity matrix of all zeros, and the repair. [5]

3 The InfoNCE logits are cosines divided by a temperature τ. State what τ cannot change for a fixed model, and what it does change. [5]

Solutions on Lecture 24's deck.

Exercises — Lecture 23 (continued)

4 Recall@10 is reported over 200 candidates. State the exact random baseline, and why it is computed rather than simulated. [3]

5 An entry with no written description is never returned by a text index built over the descriptions. State whether this is a ranking failure or a structural one, and what follows. [4]

Solutions on Lecture 24’s deck.

THE FIVE SET AT THE END OF LECTURE 22

Solutions Lecture 22

not part of this lecture

Solutions — Lecture 22

1. A recommender is scored by RMSE on held-out ratings. State why that is the wrong metric, using the set the…

✓ RMSE is computed on films the user chose to watch and rate; a recommender operates on the complement.

- It weights every prediction equally when only ten items are shown
- A constant offset ruins RMSE and changes no ranking at all

2. The same model scores HR@10 of 0.0926 and 0.8102 on the same data. Name the two decisions that differ, and…

✓ Random against temporal split, and sampled-100 against the full catalogue. Sampling moves it far more.

- Ten of 101 candidates is already a tenth of the set
- A random ranker goes from 0.0027 to 0.0990 under the same change

Solutions — Lecture 22 (2)

3. Explain why a sampled metric is not merely noisier than a full-catalogue one, and give the consequence for…

✓ The 100 negatives are drawn uniformly from a mostly-tail catalogue, so the distortion depends on the model. Orderings can reverse.

- A popularity model competes against almost no popular items
- A uniform bias would at least preserve rankings; this one does not

4. A new film arrives with a genre and a description but no interactions. State what a matrix factorisation can…

✓ The factorisation has no vector for it at all; a feature tower encodes its genre and text, so it has a vector at once.

- Matrix factorisation is the two-tower model whose towers are lookup tables, and there is no row to look up for an item never interacted with
- The same argument covers the tail: a feature tower shares parameters with similar items, where a lookup row is estimated from very few interactions

Solutions — Lecture 22 (3)

5. A recommender is trained on interactions produced by an earlier recommender. Name the effect, and say why no…

✓ The feedback loop. The log records only what the earlier system chose to show, so it rewards reproducing that system.

- Users cannot interact with what they were not shown
- A model that would have shown something better scores badly, because the evidence for it was never collected; only an online test can see it

LECTURE 24 · GÉRON CH. 15–16

Generation, retrieval-augmented systems, and where this leaves you

Captioning the catalogue entries that have no usable description

Retrieval-augmented generation: Lecture 20’s retriever joined to Lecture 18’s generator, and the failures that belong to the join

The course as one argument, and what it did not cover

The last lecture